```
# Chatbot Implementation
import nltk
from nltk.chat.util import Chat, reflections

pairs = [
    [r"hi|hey|hello", ["Hello! I am an NLTK retrieval-based chatbot."]],
    [r"what is nlp?", ["Natural Language Processing allows computers to understand hu
    [r"quit", ["Goodbye!"]]
]

chatbot = Chat(pairs, reflections)
print("User: hi")
print("Bot:", chatbot.respond("hi"))
print("User: what is nlp?")
print("Bot:", chatbot.respond("what is nlp?"))
```

```
User: hi
Bot: Hello! I am an NLTK retrieval-based chatbot.
User: what is nlp?
Bot: Natural Language Processing allows computers to understand human interactions.
```

```
# Question Answering Implementation
from transformers import pipeline

qa_pipeline = pipeline("question-answering", model="distilbert-base-cased-distilled-s
context = "Natural Language Understanding (NLU) takes a natural language input and pr
question = "What does NLU produce?"

answer = qa_pipeline(question=question, context=context)
print(answer)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarni
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (http
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public m
  warnings.warn(
config.json: 100%                                                 473/473 [00:00<00:00, 8.80kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated request
model.safetensors: 100%                                           261M/261M [00:03<00:00, 157MB/s]
Loading weights: 100%                 102/102 [00:00<00:00, 308.25it/s, Materializing param=qa_outputs.weight]
tokenizer_config.json: 100%                                       49.0/49.0 [00:00<00:00, 1.62kB/s]
vocab.txt:        213k/? [00:00<00:00, 3.62MB/s]
tokenizer.json:        436k/? [00:00<00:00, 8.82MB/s]
{'score': 0.516136884689331, 'start': 0, 'end': 30, 'answer': 'Natural Language Unde
```

```
# Summarization Implementation (Explicit)
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Load the model and tokenizer directly
model_name = "sshleifer/distilbart-cnn-12-6"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text = "Text summarization is the technique of creating a short, accurate, and flue
```

```
# 1. Tokenize the input text
inputs = tokenizer(text, return_tensors="pt", max_length=1024, truncation=True)

# 2. Generate the summary IDs
summary_ids = model.generate(inputs["input_ids"], max_length=30, min_length=10, do_

# 3. Decode the IDs back into readable text
summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
print(summary)
```

```
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 368B/s]
vocab.json:  899k/? [00:00<00:00, 7.02MB/s]
merges.txt:  456k/? [00:00<00:00, 4.68MB/s]
pytorch_model.bin: 100%  1.22G/1.22G [00:13<00:00, 95.3MB/s]
Please make sure the generation config includes `forced_bos_token_id=0`.
model.safetensors:  44%  538M/1.22G [00:09<00:10, 68.4MB/s]
Loading weights: 100%  358/358 [00:00<00:00, 623.08it/s, Materializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.shared.wei
The tied weights mapping and config for this model specifies to tie model.shared.wei
 Summarization attempts to reduce a section of text to a smaller amount . It aims to
```

```
# Machine Translation Implementation (Explicit)
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Load the T5 model and tokenizer directly
model_name = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# T5 requires a task prefix in the prompt
text_to_translate = "translate English to French: Machine translation automatically

# 1. Tokenize the input
inputs = tokenizer(text_to_translate, return_tensors="pt")

# 2. Generate the translated IDs
translated_ids = model.generate(inputs.input_ids, max_length=40)

# 3. Decode back to text
translated_text = tokenizer.decode(translated_ids[0], skip_special_tokens=True)
print(translated_text)
```

```
config.json:  1.21k/? [00:00<00:00, 22.9kB/s]
tokenizer_config.json:  2.32k/? [00:00<00:00, 11.1kB/s]
spiece.model: 100%  792k/792k [00:00<00:00, 7.69MB/s]
tokenizer.json:  1.39M/? [00:00<00:00, 13.5MB/s]
model.safetensors: 100%  242M/242M [00:02<00:00, 206MB/s]
Loading weights: 100%  131/131 [00:00<00:00, 153.61it/s, Materializing param=shared.weight]
generation_config.json: 100%  147/147 [00:00<00:00, 2.50kB/s]
La traduction automatique traduit automatiquement le texte d'une langue naturelle en
```

```
# Information Extraction (NER) Implementation
import spacy

nlp = spacy.load("en_core_web_sm")
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)

for ent in doc.ents:
    print(f"{ent.text} - {ent.label_}")
```

```
Apple - ORG
U.K. - GPE
$1 billion - MONEY
```

```
# Information Retrieval (TF-IDF) Implementation
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

documents = [
    "The child makes the dog happy",
    "The dog makes the child happy"
]

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

df = pd.DataFrame(tfidf_matrix.toarray(), columns=vectorizer.get_feature_names_out(
print(df)
```

```
      child       dog     happy     makes       the
0  0.353553  0.353553  0.353553  0.353553  0.707107
1  0.353553  0.353553  0.353553  0.353553  0.707107
```

```
# Text Classification Implementation
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

corpus = ["I need help with my bank account", "What is the weather today", "Transfe
labels = ["Banking", "Weather", "Banking", "Weather"]

vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(corpus)

classifier = MultinomialNB()
classifier.fit(X_train, labels)

test_query = ["How do I open an account"]
X_test = vectorizer.transform(test_query)

prediction = classifier.predict(X_test)
print(prediction)
```

```
['Banking']
```

```
# Semantic Search Implementation
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Local search results related to nearby pizzerias.",
    "The newest smartphone features a great camera.",
    "A recipe for making a pizza."
]
query = ["Where can I order a pizza?"]
```

```
query = ["where can I order a pizza?"]

vectorizer = TfidfVectorizer().fit(documents + query)
doc_vectors = vectorizer.transform(documents)
query_vector = vectorizer.transform(query)

similarities = cosine_similarity(query_vector, doc_vectors)
print(similarities)
```

```
[[0.         0.         0.17163517]]
```

Start coding or generate with AI.